

**Universidad Tecnológica Nacional
Facultad Regional Villa María**

TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN



TRABAJO PRÁCTICO INTEGRADOR I

Integrantes: Ankovic Angelina, Carranza Carolina, Negro Delfina, Serafini Renata

Docentes: Rodriguez Federico, Molina Eloy

Consigna 1 — Validación de usuarios y análisis de consistencia del sistema.

PARTE A — Análisis con conjuntos.

Una empresa de desarrollo analiza el comportamiento de los usuarios de su plataforma. Se cuenta con registros de IDs de usuarios según distintas actividades.

Se dispone de los siguientes datos:

$A = [101, 102, 103, 104, 105, 106]$ $B = [104, 105, 106, 107, 108]$ $C = [102, 105, 109]$

donde:

- A: usuarios que acceden a través de la API
- B: usuarios que acceden a través de la web
- C: usuarios que han generado errores

1. Calcular e interpretar:

- Usuarios que utilizan ambas plataformas:
 $A \cap B = \{104, 105, 106\}$
- Usuarios que utilizan al menos una plataforma:
 $A \cup B = \{101, 102, 103, 104, 105, 106, 107, 108\}$
- Usuarios que utilizan la plataforma, pero no presentan errores:
 $(A \cup B) - C = \{101, 103, 104, 106, 107, 108\}$
- Usuarios que utilizan exclusivamente una sola plataforma:
 $A \Delta B = \{101, 102, 103, 107, 108\}$

2. Expresar al menos dos resultados usando comprensión de conjuntos:

$$A \cap B = \{x \in \mathbb{N} \mid x \in A \wedge x \in B\}$$

$$A \cup B = \{x \in \mathbb{N} \mid x \in A \vee x \in B\}$$

3. Detectar:

- Usuarios que aparecen en C pero no en A ∪ B:
a) $C - (A \cup B) = \{109\}$

Parte B — Lógica proposicional.

5. Construir la tabla de verdad:

Usuarios	p	q	r	$p \vee q$	$(p \vee q) \wedge r$
105	V	V	V	V	V
104, 106	V	V	F	V	F
102	V	F	V	V	V
101, 103	V	F	F	V	F
-	F	V	V	V	V
107, 108	F	V	F	V	F
109	F	F	V	F	F
-	F	F	F	F	F

7. Clasificar los usuarios del dataset en:

- Críticos y no críticos.

Usuarios	$(p \vee q) \wedge r$	Clasificación de Usuarios
101	F	No Crítico
102	V	Crítico
103	F	No Crítico
104	F	No Crítico
105	V	Crítico
106	F	No Crítico
107	F	No Crítico
108	F	No Crítico
109	F	No Crítico

6. Implementar una función en Python que evalúe la expresión:

Primero se definieron tres listas: *A*, *B* y *C*. La lista *A* representa los usuarios que acceden por API, la lista *B* los usuarios que acceden por web y la lista *C* los usuarios que generaron errores en el sistema. Después, se creó una lista llamada *usuarios* que contiene todos los IDs registrados. Luego, mediante un ciclo *for*, el programa recorre cada usuario de la lista para analizar su comportamiento dentro del sistema.

Para cada usuario, se crean tres variables booleanas:

- *p*: verifica si el usuario pertenece a A
- *q*: verifica si el usuario pertenece a B
- *r*: verifica si el usuario pertenece a C

Luego se evalúa la condición lógica: $(p \vee q) \wedge r$

Esto significa que un usuario será considerado crítico si utiliza la plataforma (API o web) y además generó errores en el sistema.

Finalmente, usando una estructura *if*, el programa muestra en pantalla si cada usuario es “Crítico” o “No crítico” según el resultado de la condición lógica.

Link del repositorio:

https://github.com/caarocarranza/TP_INTEGRADOR_I/blob/main/usuarios_criticos.PY

Parte C — Análisis.

8. Responder:

- ¿Qué tipo de usuario representa mayor riesgo?

Los usuarios críticos son los que representan un mayor riesgo ya que pertenecen a C (*r*), es decir, los usuarios que han generado errores (102-105). Esto puede afectar el funcionamiento del sistema, provocar fallos y generar problemas para otros usuarios.

- ¿Qué significa que un usuario esté en C pero no en $A \cup B$?

Significa que el usuario presenta errores registrados, pero no aparece utilizando ninguna de las plataformas. En este caso, es el usuario 109.

- ¿Qué decisión tomarían como equipo programador?

Como equipo programador, nosotros prestaríamos más atención a los usuarios críticos, porque son los que cumplen la condición de riesgo.

Podríamos controlar mejor sus acciones, revisar su actividad y agregar medidas de seguridad para evitar problemas en el sistema.

CONSIGNA 2 - Análisis y optimización de costos de desarrollo.

PARTE A - Análisis matemático.

1. Determinar pendiente y ordenada al origen de funciones $A(x) = 40x + 200$ y

$B(x) = 70x + 50$:

$A(x)$: Pendiente = 40

Ordenada al origen = 200

$B(x)$: Pendiente = 70

Ordenada al origen = 50

2. Indicar si son paralelas

Las funciones A y B NO son paralelas ya que tienen pendientes diferentes.

3. Calcular el punto de intersección entre $A(x) = 40x + 200$ y $B(x) = 70x + 50$:

Coordenada X:

$$40x + 200 = 70x + 50$$

$$40x - 70x = 50 - 200$$

$$-30x = -150$$

$$x = -150 / -30$$

$$x = 5$$

Coordenada Y:

$$Y = 40x + 200$$

$$y = 40 * 5 + 200$$

$$y = 200 + 200$$

$$y = 400$$

El punto de intersección de las funciones A y B se encuentra en las coordenadas $x=5$, $y=400$

4. Función $C(x) = -2x^2 + 80x + 100$

$$a = -2$$

$$b = 80$$

$$c = 100$$

Calcular vértice:

$$Xv = \frac{-b}{2a}$$

$$Xv = \frac{-80}{2 * (-2)}$$

$$Xv = \frac{-80}{-4} \gg Xv = 20$$

$$Yv = -2 * (20)^2 + 80 * (20) + 100$$

$$Yv = -2 * (400) + 80 * (20) + 100$$

$$Yv = -800 + 1600 + 100 \gg Yv = 900$$

Xv= 20, Yv=900

Calcular raíces:

$$x = -b \pm \frac{\sqrt{b^2 - 4ac}}{2a}$$

$$x = -80 \pm \frac{\sqrt{80^2 - 4 * (-2) * 100}}{2 * (-2)}$$

$$x = -80 \pm \frac{\sqrt{6400 - (-800)}}{-4}$$

$$x1 = -80 + \frac{\sqrt{7200}}{-4} \gg x1 = -1.21$$

$$x2 = -80 - \frac{\sqrt{7200}}{-4} \gg x2 = 41.21$$

PARTE B - Implementación.

5. Programar las funciones en python

Se realizó un programa que actúa como un sistema de información y presupuesto de costos para una empresa de desarrollo de software. El sistema inicia informando de un tope máximo de 50 horas de trabajo mensuales por cliente y despliega un menú interactivo para que el usuario elija qué plan de contratación desea analizar. Luego, solicita ingresar la cantidad de horas estimadas para calcular un presupuesto personalizado.

Los datos evaluados de cada plan son:

PLANES LINEALES (PLAN A Y B):

- **Precio por hora (Costo Variable):** Es directamente la pendiente m de la recta, fijada de forma interna por la empresa para cada plan 40 para el Plan A y 70 para el Plan B. Representa cuánto aumenta la factura por cada hora extra de trabajo.
- **Costo fijo inicial (Abono base):** Es directamente la ordenada al origen b , fijada de forma interna por la empresa 200 para el Plan A y 50 para el Plan B. Representa lo que el cliente debe abonar aunque consuma 0 horas de desarrollo.
- **Presupuesto Personalizado:** El programa toma las horas ingresadas por el usuario, valida que estén dentro del dominio permitido, de 0 a 50 horas para los planes A y B, o de cero al límite en donde el valor se vuelve cero para el plan C, aplica la función correspondiente para calcular el Costo Total, mostrando además los detalles necesarios a conocer.

PLAN CUADRÁTICO C:

- **Costo fijo inicial:** Es el valor del término independiente ($c = 100$) dentro de la ecuación. Representa la tarifa básica base que la empresa le cobra al cliente al momento de contratar el plan, incluso si ese mes registra 0 horas de desarrollo.
- **Precio por hora:** Representa el coeficiente lineal b de la ecuación. Representa el valor base antes de que comiencen a aplicarse los descuentos automáticos por el volumen de trabajo.
- **Límite comercial (Raíz):** Para este plan, el programa calcula la raíz real positiva evaluando el discriminante de la ecuación. Esta raíz representa el punto exacto donde la curva cae y toca el eje X (costo cero). Como un costo negativo no tiene sentido comercial (la empresa trabajaría a pérdida o le pagaría al cliente), el programa utiliza el valor de esta raíz (41.23 horas) como un límite para recortar el plan y bloquear cualquier ingreso superior.
- **Punto máximo de costo:** Se calcula mediante el vértice de la parábola. El sistema utiliza la fórmula del vértice en X para determinar que a las 20 horas de trabajo se alcanza el tope de facturación del plan, e informa a través del vértice en Y que dicho costo máximo es de exactamente \$900. A partir de ese punto, el componente cuadrático negativo empieza a actuar como un descuento a favor del cliente.

SEGUNDO MENÚ INTERACTIVO:

Se inicia con un bucle $\text{while} \neq 3$ para que el menú siga apareciendo mientras el usuario siga eligiendo otras opciones

1. **Generar gráfico de la función:** Para ello se implementaron librerías de python
2. **Obtener valor de Y ingresando valor de X:** Se pide valor de x y se reemplaza x de la ecuación generada por el usuario por el nuevo valor.
3. **Salir del programa:** Sale del bucle while ya que genera la condición $\text{while} == 3$.

RECURSOS UTILIZADOS:

El programa fue desarrollado usando bloques if - elif - else para estructurar la toma de decisiones del sistema según el plan elegido por el usuario y para validar que las horas ingresadas respeten los límites comerciales. Se usaron bloques while para controlar los submenús de navegación gráfica sin que el programa se cierre abruptamente.

En esta actividad se utilizó la Inteligencia artificial Gemini para pulir la lógica financiera que el programa debía seguir (evitando los costos negativos del Plan C mediante el cálculo automatizado de su raíz), y para la comprensión e implementación de bibliotecas de generación de gráficos de funciones de python.

Bibliotecas utilizadas:

numpy:

Esta librería es muy útil en el manejo de datos numéricos masivos. Se utilizó para generar de forma matemática el rango y una lista de 100 números continuos para el eje X. Su ventaja es la velocidad y el uso en conjunto con la librería matplotlib para generar gráficos.

matplotlib.pyplot:

El módulo .pyplot indica que se usa esa subcarpeta específica para dibujos simples de dos dimensiones dentro de la gran librería matplotlib.

Es una biblioteca muy usada para graficar funciones matemáticas. Agarra los puntos generados por la biblioteca numpy, los dibuja en la pantalla y los une con una línea para formar el gráfico final.

Abre una ventana que permite moverse por los ejes, hacer zoom o guardar el gráfico como una foto.

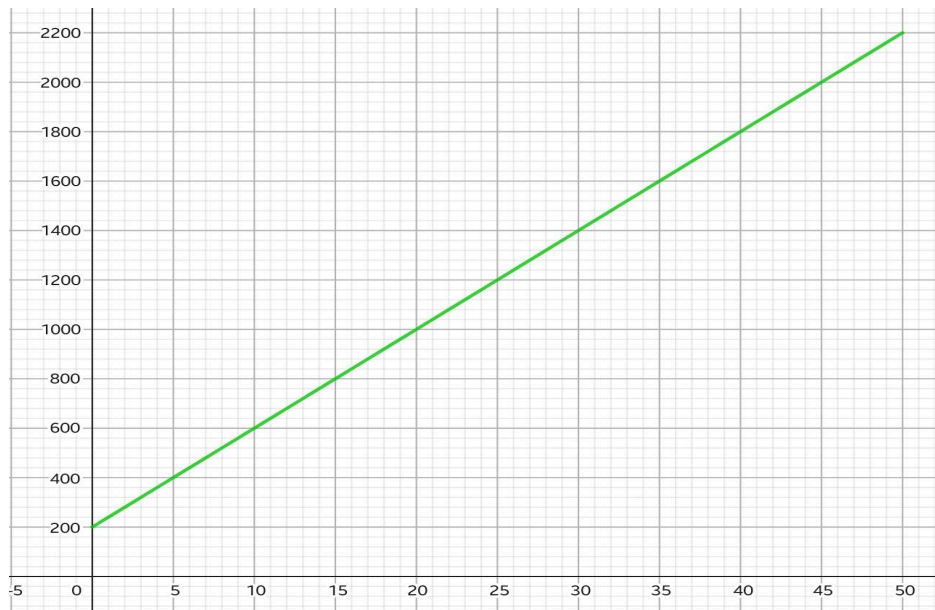
Se eligió esta porque era de las más sencillas de implementar en programas sencillos como el que requería.

Link del repositorio:

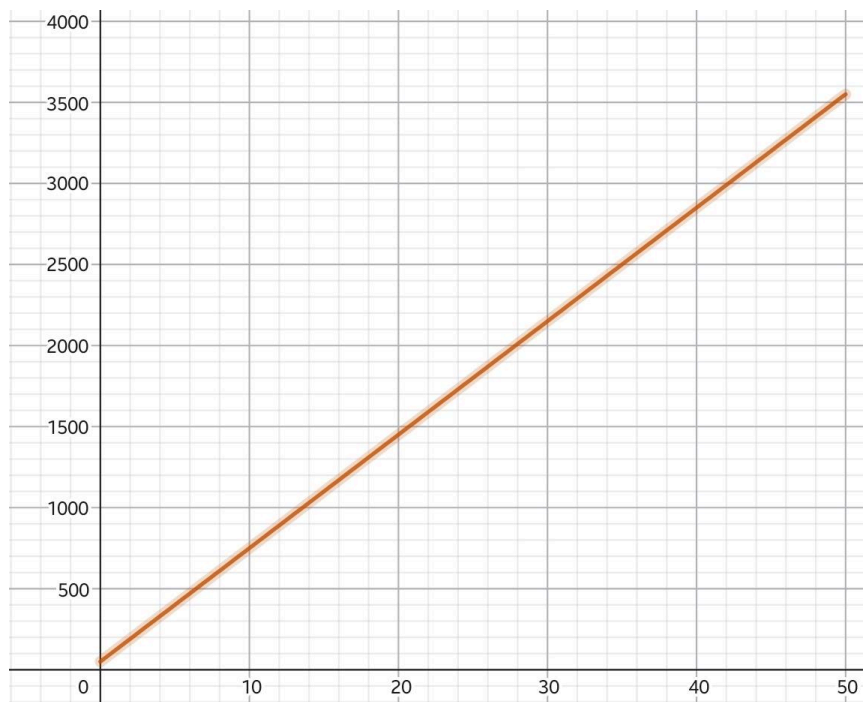
https://github.com/caarocarranza/TP_INTEGRADOR_I/blob/angelina/consigna2.5NUEVA.py

6. Graficar en el intervalo dado:

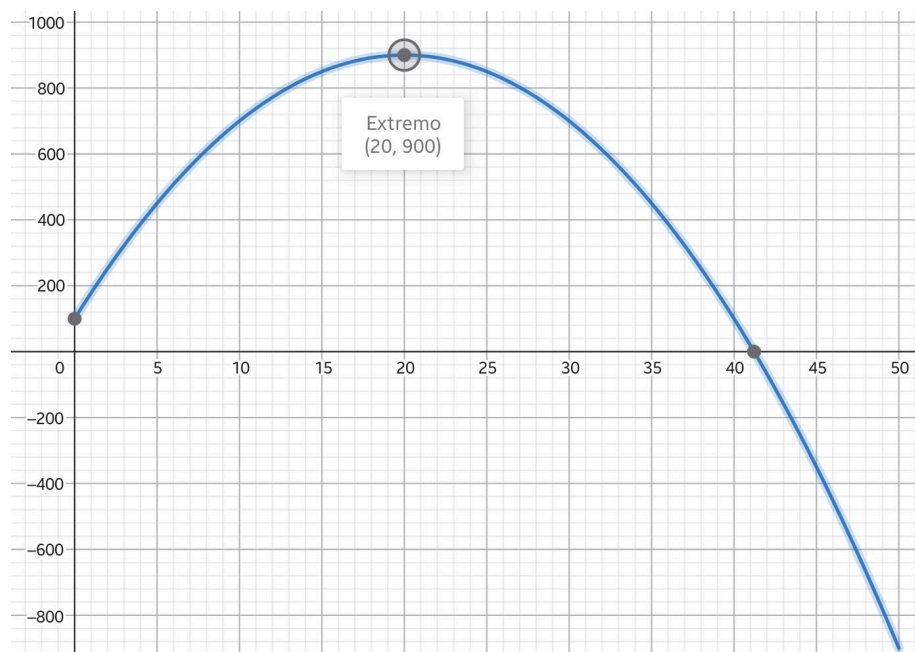
$A(x) = 40x + 200$ en un intervalo de $0 \leq x \leq 50$



$B(x) = 70x + 50$ en un intervalo de $0 \leq x \leq 50$



$C(x) = -2x^2 + 80x + 100$ en un intervalo de $0 \leq x \leq 50$



7. Evaluar las funciones para los valores: $x = [0, 5, 10, 15, 20, 25, 30, 40, 50]$

$$A(x) = 40x + 200$$

x	$A(x)$
0	200
5	400
10	600
15	800
20	1000
25	1200
30	1400
40	1800
50	2200

$$B(x) = 70x + 50$$

x	$B(x)$
0	50
5	400
10	750
15	1100
20	1450
25	1800
30	2150
40	2850
50	3550

$$C(x) = -2x^2 + 80x + 100$$

x	$C(x)$
0	100
5	450
10	700
15	850
20	900
25	850
30	700
40	100
50	-900

8. Crear una función que determine el plan más económico para un valor de x.

Para crear este programa se utilizó el análisis gráfico de la consigna siguiente. En lugar de hacer que el programa calcule y compare las tres fórmulas matemáticas cada vez que se ingresa un dato, se programaron directamente con los límites interpretados.

El programa inicia pidiendo al usuario la cantidad de horas que desea contratar, luego, mediante estructuras if - elif - else, se evalúa el número (horas) para mostrarle el plan conveniente junto con su costo total que se calcula previamente con la fórmula matemática correspondiente a cada plan:

- Si está entre 0 y 5, muestra el costo calculado del Plan B.
- Si está entre 5 y 17, muestra el costo del Plan A.
- Si está entre 17 y 41.2, muestra el costo del Plan C.
- Si está entre 41.2 y 50 horas, el programa informa que el plan C ya no está disponible para evitar costos negativos y le ofrece el plan A, que se convierte en el más conveniente nuevamente.
- Si el número ingresado no está entre el 0 y el 50, se informa que el número ingresado no es válido.

Por último, con un bloque try - except con ValueError se evita que el programa se cierre si el usuario ingresa otro valor que no sea numérico

Link repositorio:

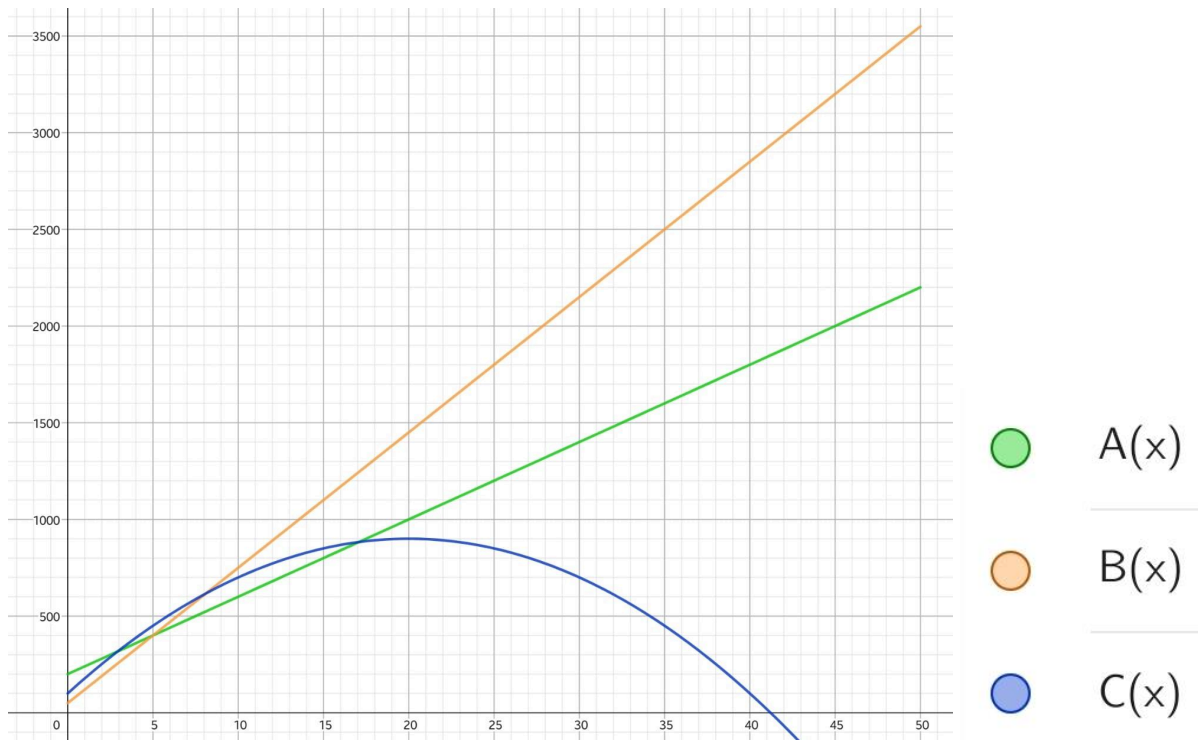
https://github.com/caarocarranza/TP_INTEGRADOR_I/blob/angelina/consigna2.8.py

PARTE C - Análisis.

9. Determinar para qué valores conviene cada plan.

Mediante el análisis de la superposición de las 3 funciones de cada plan se puede analizar que:

- **De 0 a 5 horas conviene el Plan B:** Para pocas horas conviene ya que tiene un costo fijo inicial más barato que las otras. En la hora 5 ya intersecta con el plan A.
- **De 5 a 17 horas conviene el Plan A:** Entre estas horas es el plan que se mantiene más abajo en cuanto a costo total a pesar de haber empezado con el costo inicial más alto.
- **De 17 a 41.2 horas conviene el Plan C:** Después de las 17 horas, mientras que el plan A sigue aumentando su precio linealmente, El plan C decrece de forma cuadrática debido a los descuentos por hora acumulada
- **De 41.2 a 50 horas conviene el plan A:** 41.2 horas es el límite del plan C, por lo que en este momento, el plan más barato se vuelve el plan A.



10. Detectar:

- **Valores donde el costo de C es negativo**

El costo del plan C se vuelve negativo para cualquier valor de horas mayor a 41.2, hasta llegar al tope de 50 horas. Este valor sale de la segunda raíz real de la función matemática, donde la parábola sigue por debajo del eje x.

- **Explicar por qué eso es un problema real**

El plan C se vuelve un problema real ya que un costo negativo en una gran cantidad de horas implicaría prácticamente tener que pagarle al cliente por las horas que trabajó el desarrollador y eso no tendría sentido en una empresa.

Consigna 3 — Análisis de rendimiento en un sistema distribuido.

Una empresa de tecnología opera un sistema distribuido donde distintas funcionalidades del backend se ejecutan en múltiples servidores. Con el objetivo de analizar el rendimiento y detectar posibles cuellos de botella, se registran métricas de ejecución.

Se dispone de la siguiente matriz de tiempos promedio de ejecución (en milisegundos):

$$M = \begin{pmatrix} 120 & 150 & 100 \\ 200 & 180 & 220 \\ 90 & 110 & 95 \end{pmatrix}$$

En esta matriz:

- Cada fila representa una función del sistema (por ejemplo: autenticación, procesamiento de datos, generación de reportes).
- Cada columna representa un servidor distinto.
- Cada valor indica el tiempo promedio de ejecución de una función en un servidor determinado.

Además, se cuenta con la siguiente matriz:

$$C = \begin{pmatrix} 30 & 20 & 10 \\ 15 & 25 & 20 \\ 40 & 10 & 30 \end{pmatrix}$$

donde cada valor representa la cantidad de ejecuciones registradas de cada función en cada servidor durante un período de tiempo.

Parte A — Comprensión del modelo.

1. Interpretar y explicar con sus palabras:

- Qué representa cada fila de M

Cada fila de la matriz (M) representa una función o proceso del sistema distribuido. Una fila completa indica cuánto tarda esa misma función en cada servidor, lo que sirve para ver en cuál corre mejor.

- Qué representa cada columna de M

Cada columna representa un servidor distinto. Si se observan de arriba a abajo, muestran cuánto tardan las diferentes funciones en ejecutarse en un mismo servidor, determinando si funciona correctamente o si se está quedando lenta.

- Qué representa un valor cualquiera de la matriz

Cada valor de la matriz (M) representa el tiempo promedio de ejecución medido en milisegundos, de una función en un servidor específico. Por ejemplo, el número 120 que está en la primera fila y primera columna, significa que la Función 1 tarda un promedio de 120 milisegundos en ejecutarse dentro del Servidor 1.

2. Analizar las dimensiones de M y C y determinar si es posible calcular el producto $M * C$. Justificar matemáticamente.

Ambas matrices tienen 3 filas y 3 columnas. La dimensión de la primera (M) es de (3×3) y la de la segunda (C) es de (3×3) . Por lo tanto, es posible calcular $(M * C)$ ya que para multiplicar matrices, la cantidad de columnas de la primera debe ser igual a la cantidad de filas de la segunda. El resultado será una nueva matriz de dimensión (3×3) .

Parte B — Implementación en Python.

3. Calcular:

- El tiempo promedio de ejecución por función
- El tiempo promedio de ejecución por servidor

Primero se creó la matriz (M), donde cada fila representa una función del sistema y cada columna representa un servidor. Los valores guardados en la matriz indican los tiempos promedio de ejecución en milisegundos.

Luego, para calcular el promedio por función, se utilizó un ciclo **for** que recorre cada fila de la matriz. Dentro de ese recorrido, otro ciclo suma todos los valores de la fila y después divide el resultado entre 3 para obtener el promedio de ejecución de cada función. Finalmente, el programa muestra el promedio obtenido en pantalla.

Después, se calculó el promedio por servidor. En este caso, el programa recorre las columnas de la matriz. Para cada columna, se suman los tiempos de todas las funciones en ese servidor y luego se divide entre 3 para obtener el promedio correspondiente.

Por último, se calculó la matriz transpuesta. Para hacerlo, primero se creó una nueva matriz vacía llamada MT . Luego, usando dos ciclos **for**, se intercambiaron las filas por columnas, asignando cada valor de la matriz original en la posición inversa dentro de la nueva matriz. Finalmente, se imprimió la matriz transpuesta en pantalla.

Link del repositorio:

https://github.com/caarocarranza/TP_INTEGRADOR_I/blob/main/promedio_ejecuci%C3%B3n.py

4. Calcular la matriz transpuesta de M y explicar qué representa en este contexto.

En la matriz original (M) las filas representan funciones y las columnas servidores. En la matriz transpuesta (MT) sucede al revés, las filas pasan a representar servidores mientras que las columnas pasan a representar funciones. Esto sirve para analizar de otra manera el rendimiento de cada servidor respecto a todas las funciones del sistema.

$$M^t = \begin{pmatrix} 120 & 200 & 90 \\ 150 & 180 & 110 \\ 100 & 220 & 95 \end{pmatrix}$$

Parte C — Análisis mediante producto matricial.

5. Calcular el producto $T=M*C$

$$\begin{pmatrix} 120 & 150 & 100 \\ 200 & 180 & 220 \\ 90 & 110 & 95 \end{pmatrix} * \begin{pmatrix} 30 & 20 & 10 \\ 15 & 25 & 20 \\ 40 & 10 & 30 \end{pmatrix} = \begin{pmatrix} 120*30 + 150*15 + 100*40 & 120*20 + 150*25 + 100*10 & 120*10 + 150*20 + 100*30 \\ 200*30 + 180*15 + 220*40 & 200*20 + 180*25 + 220*10 & 200*10 + 180*20 + 220*30 \\ 90*30 + 110*15 + 95*40 & 90*20 + 110*25 + 95*10 & 90*10 + 110*20 + 95*30 \end{pmatrix} =$$
$$T = \begin{pmatrix} 9850 & 7150 & 7200 \\ 17500 & 10700 & 12200 \\ 8150 & 5500 & 5950 \end{pmatrix}$$

Parte D — Interpretación crítica.

6. Analizar el resultado obtenido:

- ¿Qué podría representar cada valor de la matriz T?

Cada valor de la matriz (T) representa una combinación entre el tiempo que tarda una función y la cantidad de veces que se ejecuta. En teoría, el resultado podría interpretarse como una aproximación del tiempo total de procesamiento del sistema.

- ¿Tiene sentido físico o práctico esta operación en el contexto planteado?

La regla para multiplicar matrices nos obliga a cruzar las filas de la primera matriz con las columnas de la segunda. En nuestro caso, eso hace que terminemos multiplicando el tiempo que tarda una función en el Servidor 2 por las ejecuciones que tuvo otra función diferente en el Servidor 1, y luego sumamos todo. Como estamos mezclando datos de servidores distintos en la misma operación, el número final no representa ninguna métrica real ni útil para medir el rendimiento.

7. En caso de considerar que el producto no representa correctamente una magnitud útil:

- Explicar por qué

Matemáticamente la operación es válida, pero los valores obtenidos no representan directamente un tiempo real, un promedio ni una cantidad exacta de ejecuciones dentro del sistema. Por eso, los resultados pueden ser difíciles de interpretar desde un punto de vista práctico o físico.

- Proponer una alternativa más adecuada para combinar la información de tiempo y cantidad de ejecuciones (por ejemplo: otra operación o enfoque)

Una alternativa más adecuada sería multiplicar cada tiempo promedio de ejecución por la cantidad de ejecuciones correspondiente en el mismo servidor y función. Por ejemplo, el Servidor 1 debería multiplicarse exclusivamente por las ejecuciones de esa misma función en el Servidor 1. De esta manera, se obtendría el tiempo total consumido por cada proceso dentro del sistema, ya que permitiría identificar qué funciones consumen más recursos, cuáles generan mayor carga en los servidores y qué partes del sistema necesitan ser optimizadas.

Otra alternativa puede ser en Python, programando un bucle que multiplique de forma directa cada celda de la matriz de tiempos por la celda que ocupa el mismo lugar en la matriz de ejecuciones. De esta manera, la matriz resultante sí nos daría como resultado los tiempos totales reales de procesamiento.

Parte E — Propiedades de la matriz.

8. Determinar si la matriz **M** es simétrica. Justificar.

Para que una matriz sea simétrica, se tiene que cumplir que sea igual a su transpuesta. Es decir, los números se tienen que reflejar como en un espejo respecto a la diagonal principal. Al comparar los elementos posición por posición, vemos que no son iguales. Por lo tanto, la matriz (**M**) no es simétrica.

$$M = \begin{pmatrix} 120 & 150 & 100 \\ 200 & 180 & 220 \\ 90 & 110 & 95 \end{pmatrix} \quad M^t = \begin{pmatrix} 120 & 200 & 100 \\ 150 & 180 & 220 \\ 90 & 110 & 95 \end{pmatrix}$$

9. Evaluar si la matriz es invertible:

- Calcular su determinante

Para encontrar el determinante de la matriz se aplica el método de Sarrus, que consiste en tomar los valores de la primera y segunda fila y agregarlos por debajo de la matriz 3x3. Esta serie de pasos da como resultado una matriz de 5x3, en la cual se calculan los productos de las diagonales principales y de las diagonales secundarias, restando el total de las secundarias al total de las principales.

Diagonales principales:

$$(120 \times 180 \times 95) = 2.052.000$$

$$(150 \times 220 \times 90) = 2.970.000$$

$$(100 \times 200 \times 110) = 2.200.000$$

$$\text{Suma: } 2.052.000 + 2.970.000 + 2.200.000 = 7.222.000$$

Diagonales secundarias:

$$(100 \times 180 \times 90) = 1.620.000$$

$$(120 \times 220 \times 110) = 2.904.000$$

$$(150 \times 200 \times 95) = 2.850.000$$

$$\text{Suma: } 1.620.000 + 2.904.000 + 2.850.000 = 7.374.000$$

$$\text{Determinante (M)} = 7.222.000 - 7.374.000 = -152.000$$

- Indicar si es posible obtener la inversa

Sí, es posible obtener la inversa. Una matriz es invertible si y sólo si su determinante es distinto de cero, como nuestro determinante dio -152.000, confirmamos que la matriz (M) sí tiene inversa.

10. Interpretar:

- ¿Qué implicaría que la matriz no sea invertible en términos de la información que representa?

Si la matriz no fuese invertible, significaría que parte de la información es dependiente o redundante, es decir, que algunos datos se repiten o dependen de otros.

En este contexto, podría indicar que algunos servidores tienen comportamientos muy parecidos y no aportan nueva información. Además, si no fuese posible calcular la matriz inversa, no se podrían realizar ciertos cálculos y análisis sobre el rendimiento del sistema.

Parte F — Análisis aplicado.

11. Determinar:

- Qué función presenta mayor costo computacional promedio

La función que presenta el mayor costo computacional promedio es la Función 2, ya que posee un tiempo promedio de ejecución de 200 milisegundos. Esto indica que es la que más tarda en procesarse y la que más recursos consume dentro del sistema.

Promedio por función:

Función 1 = 123.33 milisegundos

Función 2 = 200.00 milisegundos

Función 3 = 98.33 milisegundos

- Qué servidor resulta más eficiente

Por otro lado, el servidor más eficiente es el Servidor 1 porque tiene un menor tiempo de ejecución de 136,67 milisegundos. Por lo tanto, ejecutará las funciones más rápido que los demás servidores ofreciendo un mejor rendimiento general.

Promedio por servidor:

Servidor 1 = 136.67 milisegundos

Servidor 2 = 146.67 milisegundos

Servidor 3 = 138.34 milisegundos

12. Proponer una recomendación técnica basada en los resultados obtenidos (por ejemplo: redistribución de carga, optimización de funciones, etc.):

Basándonos en los promedios calculados, el equipo propone dos recomendaciones:

La primera consiste en revisar el código de la Función 2. Debido a que es la que más tiempo tarda (200.00 ms), el objetivo sería analizar sus bucles o condiciones en Python para ver si se puede simplificar el código y lograr que se ejecute más rápido.

La segunda recomendación consiste en reasignar las tareas pesadas al servidor más rápido. Los resultados indican que el Servidor 1 es el más rápido de todos (136.67 ms) y el Servidor 2 es el más lento (146.67 ms). Por lo tanto, sugerimos redistribuir las tareas: pasar la mayor parte de las ejecuciones de la Función 2 (que es la más pesada) al Servidor 1. De esta manera, se aprovecharía la computadora que mejor rinde para procesar el trabajo más difícil, aliviando al servidor más lento.

Link repositorio: https://github.com/caarocarranza/TP_INTEGRADOR_I.git

Link video: <https://youtu.be/6ZiuAaK93vY>